RIVER VALLEY HIGH SCHOOL
General Certificate of Education Advanced Level
Higher 2
JC2 Common Test 3

COMPUTING

Paper 2 (Lab-based)

9569/02
1 JULY 2022
3 hours

Additional Materials:

Electronic version of:

- `Supermarket_items.txt`
- `Task1.ipynb`
- `Task2.ipynb`
- `Task3.ipynb`
- `Data_files` folder consists of:
  - `CustomerInfo.csv`
  - `MonitorInfo.csv`
  - `ProductInfo.csv`
  - `SalesRecord.csv`
  - `For_task4_5` folder consists of:
    - `check_digit_tables.txt`
    - `error_messages.txt`

Insert Quick Reference Guide

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of **100** will be awarded for the use of common coding standards of programming style.

The number of marks is given in brackets [ ] at the end of each question or part question. The total number of marks for this paper is 100.

This document consists of **15** printed pages.

Instruction to candidates:

Your program code and output for each of Task 1 to 3 should be downloaded in a single `.ipynb` file. For example, your program code and output for Task 1 should be downloaded as

`TASK1_<your name>_<centre number>_<index number>.ipynb`

1 The task is to perform sorting on a 2D array.
For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        #Program code
        print("Task 1.1")

        Task 1.1

In [2]: #Task 1.1
        #Program code
        print("Task 1.2")

        Task 1.2
```

Task 1.1

Write a function `gernerate_2D_numbers(n)` that returns a 2D array of integers. [4]

The function should:

- create a 2D array of size `n*n`
- assign random integers (need not be unique) with range from `10` to `90` to all cells of the 2D array
- return the 2D array

Test your function with the following code:

```
lst_2d = gernerate_2D_numbers(9)
for record in lst_2d:
    print(record)
```

An example of a correct output:

```
[35, 76, 10, 78, 77, 66, 65, 52, 45]
[20, 55, 28, 85, 15, 30, 54, 16, 64]
[79, 25, 27, 42, 58, 40, 33, 63, 32]
[71, 19, 12, 44, 43, 22, 51, 31, 38]
[82, 23, 56, 29, 13, 26, 81, 57, 34]
[60, 59, 83, 73, 70, 72, 39, 50, 69]
[17, 46, 53, 75, 11, 18, 88, 80, 47]
[41, 67, 37, 62, 87, 61, 89, 21, 90]
[84, 74, 86, 68, 14, 48, 49, 36, 24]
```

Task 1.2

Write two procedures:

- `insertion_sort(lst)`
- `insertion_sort_all_rows(lst_2d)`

The procedure `insertion_sort(lst)` should:

- take the 1D array of integer `lst` as input
- sort the integers in `lst` using insertion sort

[3]

The procedure `insertion_sort_all_rows(lst_2d)` should:

- take the 2D array of integer `lst_2D` as input
- sort every row of `lst_2D` using the function `insertion_sort(lst)`

[1]

Test your procedure with the following code:

```
lst_2d = [[48, 60, 26, 19, 79, 53, 45, 86, 50],
          [82, 29, 55, 72, 39, 62, 88, 35, 24],
          [14, 58, 85, 37, 18, 74, 20, 25, 47],
          [41, 49, 56, 61, 81, 68, 13, 70, 51],
          [43, 90, 63, 10, 65, 83, 76, 46, 87],
          [84, 67, 54, 11, 31, 17, 28, 15, 64],
          [34, 69, 73, 16, 32, 71, 89, 36, 57],
          [22, 38, 27, 66, 40, 12, 78, 23, 44],
          [75, 77, 30, 59, 21, 80, 33, 52, 42]]
insertion_sort_all_rows(lst_2d)
for result in lst_2d:
    print(result)
```

An example of a correct output:

```
[19, 26, 45, 48, 50, 53, 60, 79, 86]
[24, 29, 35, 39, 55, 62, 72, 82, 88]
[14, 18, 20, 25, 37, 47, 58, 74, 85]
[13, 41, 49, 51, 56, 61, 68, 70, 81]
[10, 43, 46, 63, 65, 76, 83, 87, 90]
[11, 15, 17, 28, 31, 54, 64, 67, 84]
[16, 32, 34, 36, 57, 69, 71, 73, 89]
[12, 22, 23, 27, 38, 40, 44, 66, 78]
[21, 30, 33, 42, 52, 59, 75, 77, 80]
```

Task 1.3

Write a procedure `bubble_sort_col_full_row(lst_2d, col)`. [6]

The procedure `bubble_sort_col_full_row(lst_2d, col)` should:

- take the 2D array of integer `lst_2d` and the integer `col` as input
- sort the rows of integers in `lst_2D` using bubble sort based on the index `col`

Test your procedure with the following code:

```
lst_2d = [[19, 26, 45, 48, 50, 53, 60, 79, 86],
          [24, 29, 35, 39, 55, 62, 72, 82, 88],
          [14, 18, 20, 25, 37, 47, 58, 74, 85],
          [13, 41, 49, 51, 56, 61, 68, 70, 81],
          [10, 43, 46, 63, 65, 76, 83, 87, 90],
          [11, 15, 17, 28, 31, 54, 64, 67, 84],
          [16, 32, 34, 36, 57, 69, 71, 73, 89],
          [12, 22, 23, 27, 38, 40, 44, 66, 78],
          [21, 30, 33, 42, 52, 59, 75, 77, 80]]

bubble_sort_col_full_row(lst_2d, 2)
for record in lst_2d:
    print(record)
```

An example of correct output, take note all the 9 rows are moved as a whole to make index `col` 2 sorted:

```
[11, 15, 17, 28, 31, 54, 64, 67, 84]
[14, 18, 20, 25, 37, 47, 58, 74, 85]
[12, 22, 23, 27, 38, 40, 44, 66, 78]
[21, 30, 33, 42, 52, 59, 75, 77, 80]
[16, 32, 34, 36, 57, 69, 71, 73, 89]
[24, 29, 35, 39, 55, 62, 72, 82, 88]
[19, 26, 45, 48, 50, 53, 60, 79, 86]
[10, 43, 46, 63, 65, 76, 83, 87, 90]
[13, 41, 49, 51, 56, 61, 68, 70, 81]
```

Task 1.4

Write a function `merge(sorted_lst_2d)` that returns a sorted 1D array of integers. [6]

The function `merge(sorted_lst_2d)` should:

- take the sorted 2D array of integer `sorted_lst_2d` as input
- perform `merge` operation (like the merge operation in merge sort) with all the sorted rows in `sorted_lst_2d`
- return a sorted 1D array of integer in ascending order

For example, when `col` is `2`:

```
sorted_lst_2d = [[14, 18, 20, 25, 37, 47, 58, 74, 85],
                 [12, 22, 23, 27, 38, 40, 44, 66, 78],
                 [11, 15, 17, 28, 31, 54, 64, 67, 84],
                 [16, 32, 34, 36, 57, 69, 71, 73, 89],
                 [24, 29, 35, 39, 55, 62, 72, 82, 88],
                 [21, 30, 33, 42, 52, 59, 75, 77, 80],
                 [19, 26, 45, 48, 50, 53, 60, 79, 86],
                 [13, 41, 49, 51, 56, 61, 68, 70, 81],
                 [10, 43, 46, 63, 65, 76, 83, 87, 90]]
print("2D array sorted by rows")
for result in sorted_lst_2d:
    print(result)
print()
print("Sorted 1D array using merge operation")
print(merge(sorted_lst_2d))
```

An example of a correct output,

```
2D array sorted by rows
[19, 26, 45, 48, 50, 53, 60, 79, 86]
[24, 29, 35, 39, 55, 62, 72, 82, 88]
[14, 18, 20, 25, 37, 47, 58, 74, 85]
[13, 41, 49, 51, 56, 61, 68, 70, 81]
[10, 43, 46, 63, 65, 76, 83, 87, 90]
[11, 15, 17, 28, 31, 54, 64, 67, 84]
[16, 32, 34, 36, 57, 69, 71, 73, 89]
[12, 22, 23, 27, 38, 40, 44, 66, 78]
[21, 30, 33, 42, 52, 59, 75, 77, 80]

Sorted 1D array using merge operation
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 2
5, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 5
6, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71,
 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 8
7, 88, 89, 90]
```

2 The task is to write a supermarket inventory menu that stores all the items information in a NoSQL database.

Task 2.1

Write code to do the following:

- create a mongo client to local host with port 27017
- create a database named "`supermarket`" and one collection named "`items`"
- read the file "`supermarket_items.txt`" and insert its information into the NoSQL database "`supermarket`" in the collection "`items`"
- the first line in the file "`supermarket_items.txt`" indicates the names of each field in the document, separated by a comma

[10]

Task 2.2

Write a menu that provides the following options. Data validation must be done for option 3. [10]

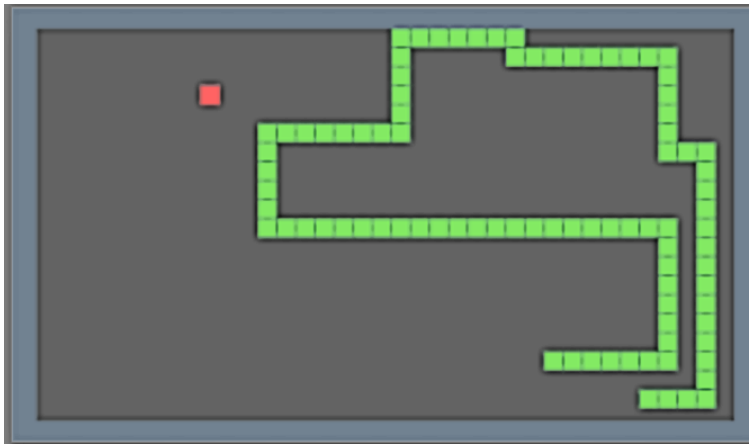
```
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
```

Test your program with the following test data. Inputs are shown in **bold**.

```
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
Choose an option:1
Type an item name:Bananas
Bananas each cost $1.5.
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
Choose an option:2
Type an item type:Fruits
1 Apples
2 Bananas
3 Berries
4 Grapes
5 Lemons
6 Lime
7 Melons
```

```
8 Nectarines
9 Oranges
10 Peaches
11 Pears
12 Plums
13 Strawberries
14 Watermelon
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
Choose an option:3
Type an item name you want to update price:Bananas
Type a new update price:9.2
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
Choose an option:1
Type an item name:Bananas
Bananas each cost $9.2.
Choose an option:
1. Find cost by item name.
2. Find items by type.
3. Update cost by item name.
4. Quit
Choose an option:4
Program quitted.
```

3 The main task is to implement the class `Snake` that partially resembles the old snake hand phone game. The objective of the snake game is to control the movement of the snake using only up, down, left and right on the gameboard. The snake will grow as it collects the food squares that are placed on the board at random empty squares.



In this task, a doubly linked list is used to represent the snake.

At one unit time, the snake head moves from one square an **adjacent empty** square based on the last movement command (diagonal movement is not allowed). To resembling this movement, a node instance with the coordinates of the adjacent square is added to head and the tail node instance is removed.

Every time the snake moves from one square to an adjacent square that contains food, its length will increase by 1. In this case, a node instance with the coordinates of the adjacent square is added to the head of the snake.

A snake is represented using a series of x and y coordinates in which each coordinate is stored in a Node instance. For example, for the snake in the board below is represented by `(8,6) (7,6) (6,6) (5,6)` and `(5,5)` where `(8,6)` is the head.

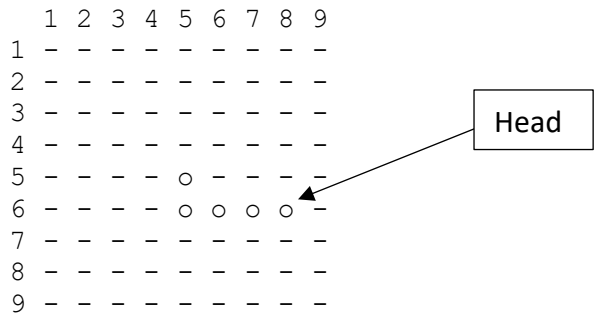


Fig. 3

Task 3.1

Write code to complete the class function `valid_pos(self, new_x, new_y)`. The constructors for both the `Node` and `Snake` class are implemented for you. You are not to edit these codes.

The function `valid_pos(self, new_x, new_y)` takes the coordinate (`new_x`, `new_y`) and checks if it is a valid coordinate for the snake to move to. It returns `True` if it is valid and `False` otherwise. The condition of a valid position is as follow:

- The new coordinates must not be occupied by the snake itself.
- The new coordinates must be one of the four coordinates surrounding the head of the snake, diagonal coordinate is not included.
- To reduce the difficulty of the question, you can assume that the game board has no border limit.

[6]

Test your function with the following code:

```
def testcase_3_1():
    s1 = Snake(5,5)
    n1 = Node(5,4)
    n2 = Node(5,3)
    n3 = Node(4,3)
    n4 = Node(4,4)
    n5 = Node(5,4)

    # creating the snake of 6 nodes.
    # head - > (5,5) (5,4) (5,3) (4,3) (4,4) (5,4) <- tail
    curr = s1.head
    for node in [n1, n2, n3, n4, n5]:
        curr.next = node
        node.prev = curr
        curr = node

    # In this case the coordinates around the head of the
    # snake are (4,5), (5,4), (5,6) and (6,5) but (5,4)
    # is part of the snake, so only (4,5), (5,6) and (6,5)
    # are valid positions to move to.

    print(s1.valid_pos(4,5))#True
    print(s1.valid_pos(5,6))#True
    print(s1.valid_pos(6,5))#True
    print(s1.valid_pos(5,4))#False

print("test case 3.1.................")
testcase_3_1()
```

Task 3.2

Write code to complete the class procedure `move(self, new_x, new_y, is_food)`.

The procedure `move(self, new_x, new_y, is_food)` takes a coordinate (`new_x`, `new_y`) and a Boolean `is_food` which indicates if the coordinate contains food.

The procedure should do the following:

- Checks if it is a valid position to move to. If it is not a valid position, there is nothing needs to be done.
- If it is a valid position, check if it contains food.
    - If it does not contain food:
        - add a node instance with the coordinates (`new_x`, `new_y`) to the head of the snake and make (`new_x`, `new_y`) as the head
        - remove the tail node instance of the snake
    - If it contains food
        - just add a node instance with the coordinates (`new_x`, `new_y`) to the head of the snake and make (`new_x`, `new_y`) as the head

[7]

Test your procedure with the following code:

```
def testcase_3_2_a():
    s1 = Snake(5,5)
    for new_x, new_y, is_food in [(5,6,False),(6,6,False),(7,6,False),(8,6,False),(9,6,False),(9,7,False),
                                  (8,7,False),(7,7,False),(6,7,False),(5,7,False),(5,6,False)]:
        s1.move(new_x, new_y, is_food)
    s1.display()
print("test case 3.2a ..................")
testcase_3_2_a()
print()

def testcase_3_2_b():
    s1 = Snake(5,5)
    for new_x, new_y, is_food in [(5,6,True),(6,6,True),(7,6,True),(8,6,True),(9,6,True),(9,7,True),
                                  (8,7,True),(7,7,True),(6,7,True),(5,7,True),(5,6,True)]:
        s1.move(new_x, new_y, is_food)
    s1.display()
print("test case 3.2b ..................")
testcase_3_2_b()
print()

def testcase_3_2_c():
    s1 = Snake(5,5)
    for new_x, new_y, is_food in [(5,6,True),(6,6,True),(7,6,True),(8,6,False), #valid
                                  (8,9,True),(7,9,True),(6,9,True),(5,9,True)]: #invalid
        s1.move(new_x, new_y, is_food)
    s1.display()
print("test case 3.2.c ..................")
testcase_3_2_c()
print()
```

Task 3.3

Write code to complete the procedure `displayboard(self, snake, symbol)` in class `Gameboard`.

The procedure `displayboard(self, snake, symbol)` takes a Snake instance `snake` and a string `symbol`. The procedure displays the snake in the game board by marking the coordinates of the full snake with the string `symbol`.

For example, given Snake instance `snake` have the coordinates of each node starting from the head as `(8,6)` `(7,6)` `(6,6)` `(5,6)` and `(5,5)`.

```
gameboard = Gameboard(9)
gameboard.displayboard(snake, "o")
```

The function should display the following format.

```
The Game Board
   1 2 3 4 5 6 7 8 9
 1 - - - - - - - - -
 2 - - - - - - - - -
 3 - - - - - - - - -
 4 - - - - - - - - -
 5 - - - - o - - - -
 6 - - - - o o o o -
 7 - - - - - - - - -
 8 - - - - - - - - -
 9 - - - - - - - - -
```

To reduce difficulty, you can assume that the maximum size of the game board is only 9.

[7]

Test your procedure with the following code:

```
def testcase_3_3_a():
    gameboard = Gameboard(9)
    snake = Snake(5,5)
    for new_x, new_y, is_food in [(5,6,True),(6,6,True),(7,6,True),(8,6,True)]:
        snake.move(new_x, new_y, is_food)
    print("The coordinates of each node starting from the head.")
    snake.display()
    gameboard.displayboard(snake, "o")


print("test case 3.3.a ..................")
testcase_3_3_a()
print()

def testcase_3_3_b():
    gameboard = Gameboard(9)
    snake = Snake(5,5)
    for new_x, new_y, is_food in [(5,6,True),(6,6,True),(7,6,True),(8,6,True),(9,6,True),(9,7,True),(9,8,True),
                                  (8,8,True),(7,8,True),(6,8,True),(5,8,True)]:
        snake.move(new_x, new_y, is_food)
    print("The coordinates of each node starting from the head.")
    snake.display()
    gameboard.displayboard(snake, "o")
print("test case 3.3.b ..................")
testcase_3_3_b()
print()
```

4 Spectrum+ is a local company selling monitors. You are tasked to implement a web application with database to aid the company for managing their sales online.

Task 4.1

The following information of each `MonitorInfo` is stored:
`ModelNo` – model number of the monitor.
`Price` – Original price of the monitor.
`Promotion` – integer storing the percentage value of promotion on going, such as `'77'` means "23% off". This value should be always larger than `'0'` and less than or equals to `'100'`.
`ScreenSize` – integer storing the size of the screen, e.g. `'24"'` implies `24` inch screen.
`Resolution` – string storing the resolution of the screen, e.g. `'1920 x 1080'`.

The following information of each `ProductInfo` is stored:
`SerialNo` – serial number following naming format of `'SPECxxxxxxxxxx'`, where `'x'` can be any alpha-numeric value from `'0'` to `'9'` and `'a'` to `'z'`.
`ModelNo` – model number of the monitor.
`Status` – status of the product, can be either `'Sold'` or `'In Stock'`.

The following information of each `CustomerInfo` is stored:
`Email` – email of the customer.
`Name` – name of the customer.
`Contact` – contact of the customer.
`Address` – address of the customer.

The following information of each `SalesRecord` is stored:
`RecordID` – auto increment integer value to keep track of the sales record.
`Email` – user email ordering the product.
`SerialNo` – serial number of the monitor sold.
`OrderDate` – date of the order, storing as a string in the format of `'YYYYMMDD'`.
`DeliveryDate` – date of the delivery, storing as a string in the format of `'YYYYMMDD'`.

Note:

- *You are not required to include check condition for the* `serialNo` *when creating the* `MonitorInfo` *table.*
- *You are required to include check condition for* `Promotion` *and* `Status` *fields.*
- *You may assume one user will order not more than one product within the same day.*

The information is to be stored in above mentioned tables:
`MonitorInfo`
`ProductInfo`
`CustomerInfo`
`SalesRecord`

Task 4.1

Create an SQL file called `Task4_1.sql` to show the SQL code to create the database `spectrum.db` with the above tables.

The table `MonitorInfo` must use `ModelNo` as its primary key, the table `ProductInfo` must use `SerialNo` as its primary key, the table `CustomerInfo` must use `Email` as its primary key, and the table `SalesRecord` must use `RecordID` as its primary key.

The `ModelNo` in table `ProductInfo` must refer to `ModelNo` in `MonitorInfo` table as foreign key. `Email` and `SerialNo` in table `SalesRecord` must refer to `Email` in `CustomerInfo` table and `SerialNo` in `ProductInfo` table as foreign keys.

Save your SQL code as
`Task4_1.sql`

[5]

Task 4.2

The following files contains the past data records. The first row of each file contains the header of the respective columns. Each row in the files is a comma-separated list of information.

```
MonitorInfo.csv
ProductInfo.csv
CustomerInfo.csv
SalesRecord.csv
```

Write a Python program to insert all information from the files into the database `spectrum.db`. Run the program.

Save your program code as
`Task4_2.py` [5]

Task 4.3

You are tasked to implement a function to count and display a summary of sales record of monitor models. Query and display a list of data with the following fields as shown in the table, sorted in the **descending** order according to the number of monitor models sold.

Note:

- *Actual Price should be calculated using the original price multiply with the **percentage** value of the promotion, and **round to 2.d.p**.*
- *Quantity refers to the number of monitors sold for each ModelNo.*

ModelNo	Actual Price	Resolution	Quantity
…	…	…	…

Write the SQL code required.

Save this code as
`Task4_3.sql` [4]

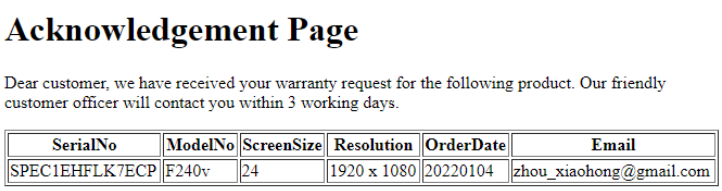
Task 4.4

You are required to implement a function for customers to submit warranty request by entering the `SerialNo` of the product. Upon receiving the `SerialNo`, query database and display an acknowledgement page with the following fields:
`SerialNo, ModelNo, ScreenSize, Resolution, OrderDate, Email`

Here's are some sample pages for your reference:

Warranty Request Page

Acknowledgement Page



Save all files and folders under the directory `Task4_4`.

Run the web application. Enter `SerialNo` with value `SPEC1EHFLK7ECP`.
Then save the output of the program as `Task4_4.html`. [12]

Task 4.5

After the internal test, the client feedback that there's a need for input validation. You are tasked to improve on Task 4.4 with the following changes:

Upon receiving the value of `SerialNo`, go through a series a validation checks to ensure the `SerialNo` entered is valid. If not valid, display a meaningful error message and ask the user to re-enter the `SerialNo`; if valid, query the database and display the acknowledgement page.

The following validation checks should be performed:

Validation Check	Error Message
Presence Check	Presence check failed. Please enter the serial no. of the product.
Length Check	Length check failed. Please make sure the serial no. has 14 characters.
Format Check1	Format check failed. Please make sure the serial no. starts with "SPEC".
Format Check2	Format check failed. Please make sure the serial no. contains only alphanumeric values.
Check digit Check	Check digit failed. Please make sure the serial no. is valid.

For your convenience and reference, the above error messages have been stored in the file `"error_messages.txt"`. *Please feel free to copy and paste to your python program file.*

The last digit of the `SerialNo` is a check digit based on the following algorithm:

- Step 1: Ignore/remove the first 4 characters `"SPEC"`.
- Step 2: Ignore/remove the last letter as it's the check digit.
- Step 3: Translate all letters to integer values using the following table:

A: 1	**B**: 2	**C**: 3	**D**: 4	**E**: 5	**F**: 6	**G**: 7	**H**: 8	**I**: 9
J: 1	**K**: 2	**L**: 3	**M**: 4	**N**: 5	**O**: 6	**P**: 7	**Q**: 8	**R**: 9
N/A	**S**: 2	**T**: 3	**U**: 4	**V**: 5	**W**: 6	**X**: 7	**Y**: 8	**Z**: 9

- Step 4: Use the following weight factor for each position.

Position	1	2	3	4	5	6	7	8	9
Weight	8	7	6	5	4	3	2	9	4

- Step 5: The sum of product of the letter/digit with their corresponding weight factor is then divided by `11`.
- Step 6: Translate the remainder from integer to an alphabet based on the following table:

0: S	**1**: P	**2**: E	**3**: C	**4**: T	**5**: R	**6**: U	**7**: M	**8**: X	**9**: Y	**10**: Z

- E.g. For `SerialNo` with value `SPEC1EHFLK7ECP`, we can first ignore the first 4 and last 1 character, which left with `1EHFLK7EC`. It will produce a check digit of `P` by going through the above algorithm. Hence `SPEC1EHFLK7ECP` is a valid `SerialNo`.

For your convenience and reference, the above conversion and weighted tables have been stored in the file `"check_digit_tables.txt"`. *Please feel free to copy and paste to your python program file.*

If `SerialNo` is invalid, display error message while direction user back to the warranty request page.

Save all files and folders under the directory `Task4_5`.

[12]

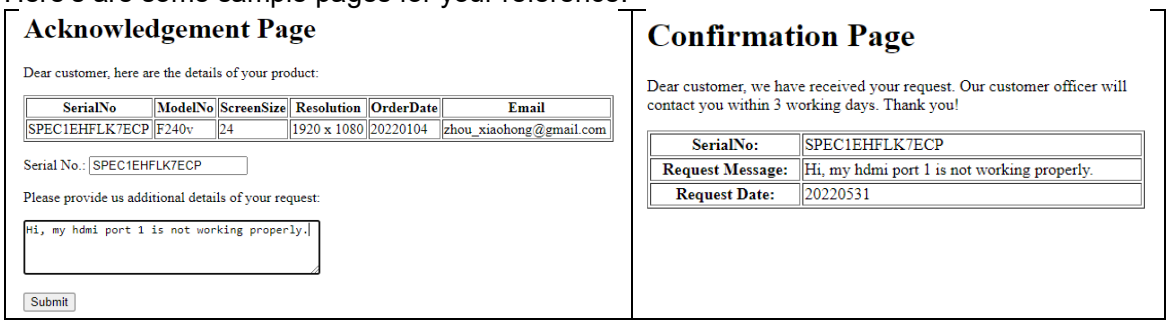
Task 4.6 [Challenge by Choice]

You are now tasked to further improve user experience with the following changes:

1. If `SerialNo` is valid, display an additional form for user to key in their request message in a textbox.
2. Store the following information into the database and display confirmation for user: `SerialNo, RequestMessage, RequestDate`
3. Date of request should be stored as the current date. Consider using the `datetime` library.
4. Create a new table inside database to reflect this change and store values submitted.

You are encouraged to duplicate the database file before making any adjustments.

Here's are some sample pages for your reference:



Save all files and folders under the directory `Task4_6`.

[2]

- End of Paper -